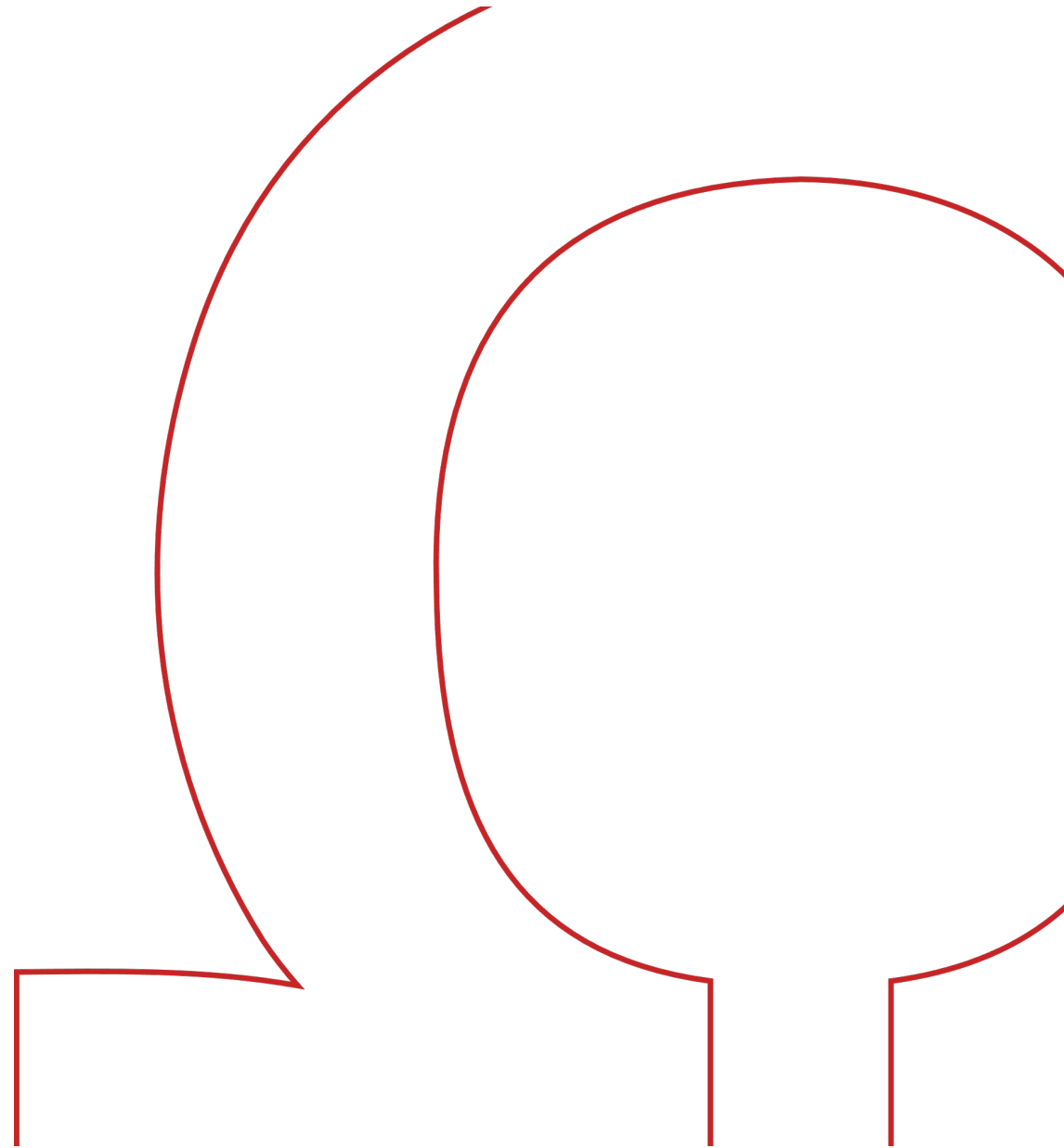


Prog III / A

Software Engineering



Kommunikation über Netzwerke

Kommunikation über Netzwerke

OSI-Modell (Open Systems Interconnection)



Sender

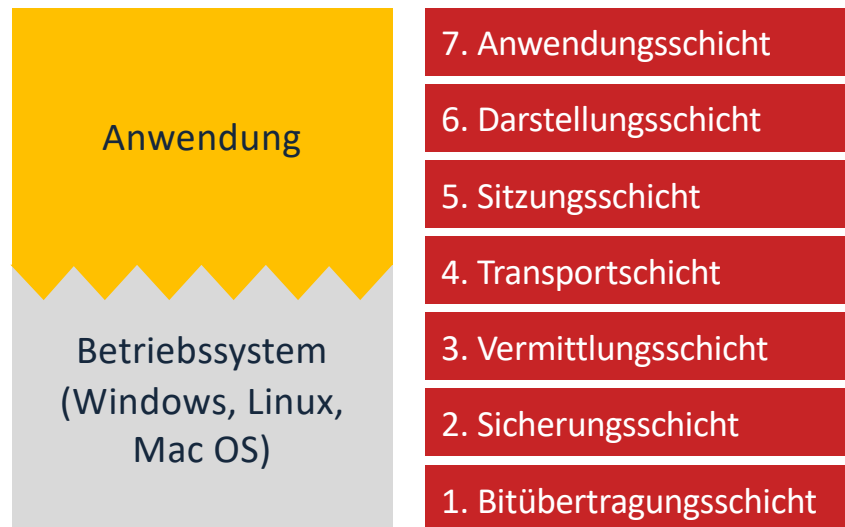


Empfänger



Kommunikation über Netzwerke

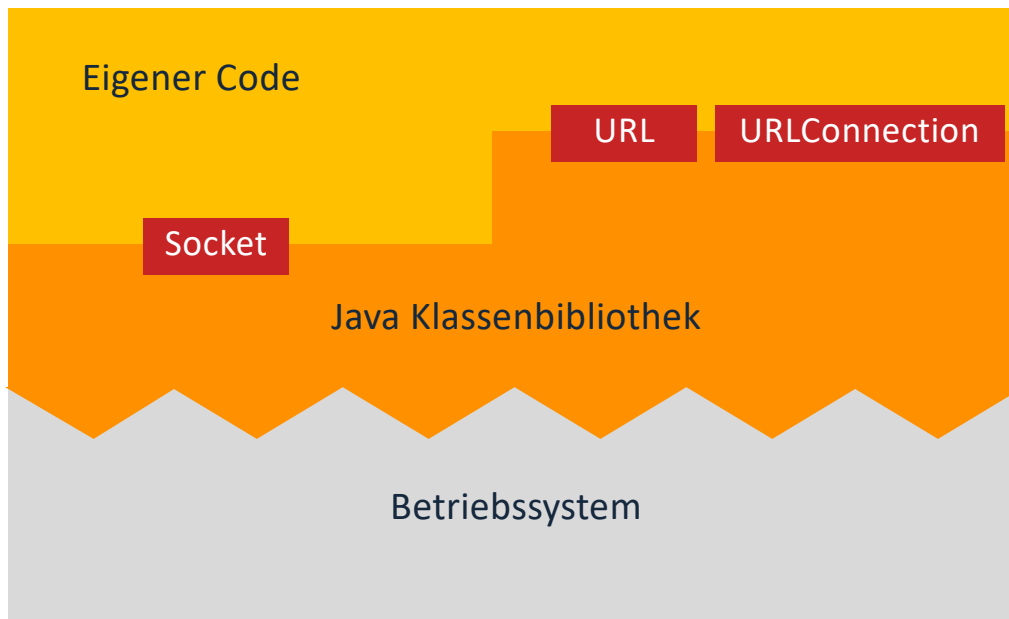
Verantwortlichkeiten



Während die unteren Schichten i.d.R. durch das Betriebssystem bereitgestellt werden, ist die jeweilige Anwendung für die Implementierung der oberen Schichten verantwortlich.

Kommunikation über Netzwerke

Netzwerkanwendungen in Java



Soll die Anwendung in Java programmiert werden, so stehen in der **Java Klassenbibliothek** einige Hilfsmittel bereit, um die Schnittstelle zum Betriebssystem einfach nutzen zu können.

Je nach Anwendungsfall kann auf unterschiedliche Klassen aufgesetzt werden:

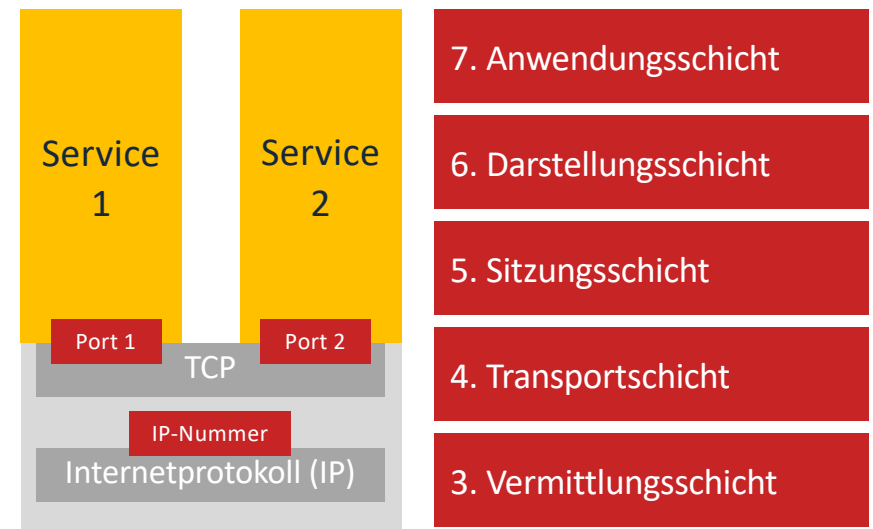
- Kommunikation mit Hilfe von TCP/IP → Klasse Socket, ...
- Kommunikation mit Hilfe von HTTP(S) → Klassen URL, URLConnection,...

TCP

TCP

Transmission Control Protocol

- Bei **TCP** baut ein **Servicenachfrager** (Client) eine Verbindung zu einem **Serviceerbringer** (Server) auf.
- Er benötigt dazu eine Adresse des Servers, die sich zusammensetzt aus der **IP-Nummer** der Servermaschine und dem **Port**, über den der Service erreichbar ist.
- Auf der Servermaschine könnten unterschiedliche Services parallel mit **verschiedenen Ports** existieren, nicht aber mit an einem Port.

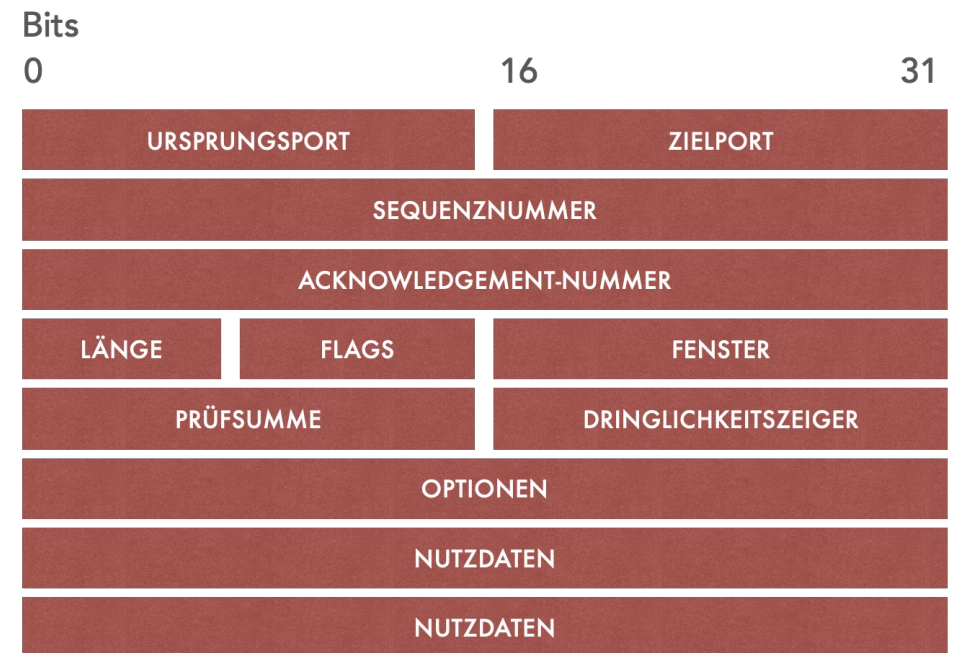


TCP

Aufbau eines TCP-Segments

Die Datensegmente, die zwischen den TCP-Schichten zweier Rechner ausgetauscht werden, haben nebenstehenden Aufbau:

- **Ursprungsport** und **Zielport** dienen der Adressierung des richtigen Service. Die richtigen Rechner werden bereits in der Schicht darunter mittels IP-Nummer identifiziert.
- Der Sender vergibt aufsteigende **Sequenznummer**, damit der Empfänger feststellen, ob ein Segment verloren gegangen ist.
- Empfangene Segmente werden dem Sender bestätigt (**Acknowledgement-Nummer**)



TCP

Ports

Der Port wird repräsentiert durch eine 16 Bit Integerzahl.

- Port 0 bis 1023: Festlegung der Services durch IANA (Internet Assigned Numbers Authority)
Services dürfen i.d.R. nur vom Admin betrieben werden
- Port 1024 bis 49151: Für diese Ports gibt es registrierte Services.
- Port 49152 bis 65535: variable Nutzung

Auswahl festgelegter/registrierter Ports

ftp	21	Senden/Empfangen von Dateien
ssh	22	Secure Shell
telnet	23	Interaktive Verbindung
smtp	25	E-Mail Store/Forward
http	80	Web-Server (unverschlüsselt)
pop3	110	E-Mail Abruf
imap	143	E-Mail Serverzugriff
ldap	389	Zugriff Benutzerverwaltung
https	443	Web-Server (verschlüsselt)
smb	445	Microsoft Netzwerkfreigaben
nfs	1025	Unix Netzwerkfreigaben
mssql	1433	Microsoft SQL-Server
civ4	2056	Civilization 4 Multiplayer
halo	2302	Halo Multiplayer
mysql	3306	MySQL Datenbank

TCP

InetSocketAddress

```
import java.net.InetSocketAddress;

public class Address {

    public static void main(String[] args) {
        InetSocketAddress address = new InetSocketAddress("localhost", 12345);
        System.out.println(address);
    }
}
```

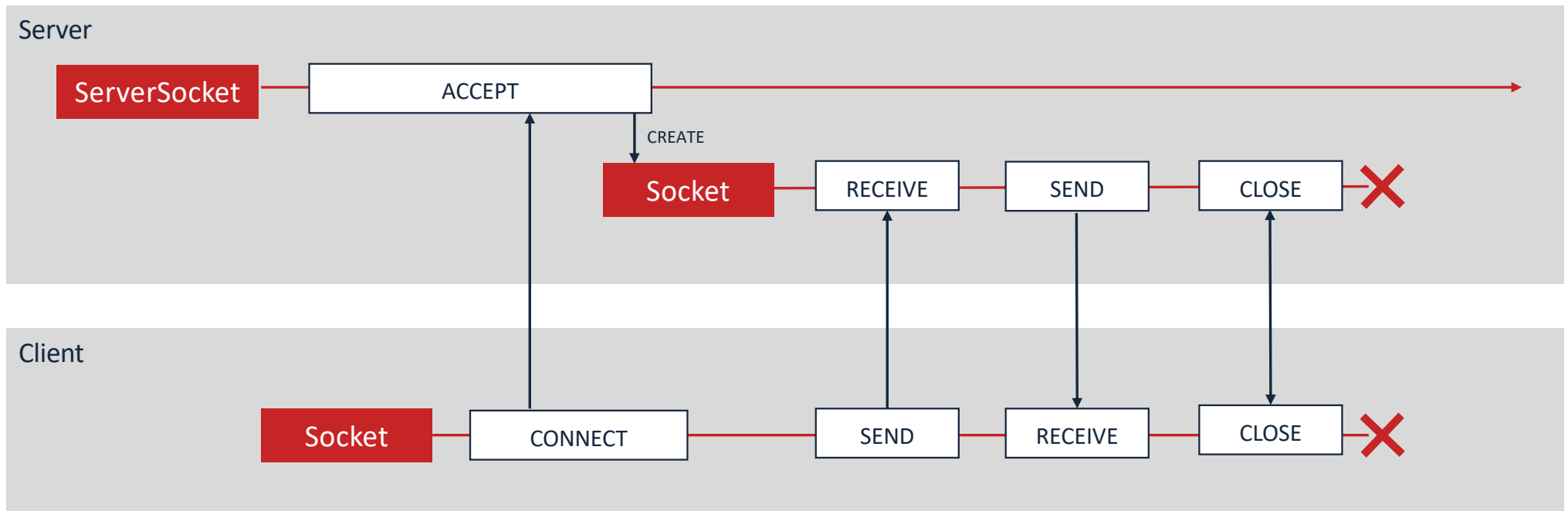


Instanzen der Klasse `InetSocketAddress` repräsentieren die vollständige Adresse eines TCP-Service bestehend auf einer Identifikation des Rechners und der Portnummer.

Client-Server-Programmierung

Client-Server-Programmierung

Kommunikationsablauf mit TCP



Client-Server-Programmierung

Server

Zunächst wird eine Instanz von `ServerSocket` mit dem Port erzeugt, auf dem gelauscht werden soll.

Der anschließende Aufruf von `accept` wartet, bis sich ein Client verbindet, und liefert eine Instanz von `Socket` zurück, die direkt mit dem `Socket` des Client verbunden ist.

Nach dem Austausch von Nachrichten müssen `Socket` und `ServerSocket` geschlossen werden.

```
import java.net.ServerSocket;
import java.net.Socket;

...

public static void main(String[] args) {

    final int port = 12345;
    try {
        ServerSocket server = new ServerSocket(port);
        Socket socket = server.accept();
        // Senden und Empfangen von Nachrichten
        socket.close();
        server.close();
    }
    catch (Exception e) {
        System.err.println("Error: " + e.getMessage());
    }
}

...
```

Client-Server-Programmierung

Server für mehrere Clients

Meist möchte man serverseitig nicht nur einen Client bedienen, sondern beliebig viele. Daher wird i.d.R. die Kommunikation in eine eigene **Handler-Klasse** ausgelagert und nach dem `accept` ein eigener Thread gestartet, der die Unterhaltung mit dem neu verbundenen Client übernimmt.

Der ursprüngliche Thread kann erneut mittels `accept` auf weitere Kontaktaufnahmen warten.

```
public static void main(String[] args) {  
  
    final int port = 12345;  
    try {  
        ServerSocket server = new ServerSocket(port);  
        while(true) {  
            Socket socket = server.accept();  
            Handler handler = new Handler(socket);  
            Thread thread = new Thread(handler);  
            thread.start();  
        }  
        server.close();  
    }  
    catch (Exception e) {  
        System.err.println("Error: " + e.getMessage());  
    }  
}
```



Client-Server-Programmierung

try mit Ressourcen

Werden in einem try-Block Ressourcen allokiert, die auch wieder freigegeben werden müssen, ist die korrekte Platzierung von `close` schwierig.

Für diesen Fall können die Ressourcen unmittelbar hinter dem try angefordert werden. Sie werden dann automatisch freigegeben, wenn der try-Block verlassen wird – sei es mit einer Exception oder dem normalen Ablauf.

Der Aufruf von `close` ist nicht mehr notwendig.

```
public static void main(String[] args) {  
  
    final int port = 12345;  
    try (ServerSocket server = new ServerSocket(port)) {  
        while(true) {  
            Socket socket = server.accept();  
            Handler handler = new Handler(socket);  
            Thread thread = new Thread(handler);  
            thread.start();  
        }  
    }  
    catch (Exception e) {  
        System.err.println("Error: " + e.getMessage());  
    }  
}
```

Client-Server-Programmierung

Serverseitiger Handler

- Die **Handler-Klasse** implementiert das Interface `Runnable`.
- Im Konstruktor wird das **Socket** für die Kommunikation mit dem Client übergeben.
- Innerhalb der dann parallel ausgeführten `run`-Methode findet die eigentliche Kommunikation mit dem Client statt.

```
class Handler implements Runnable {  
  
    private final Socket socket;  
  
    public Handler(Socket socket) {  
        this.socket = socket;  
    }  
  
    public void run() {  
        try {  
            // mit this.socket: Senden und Empfangen  
        }  
        catch (Exception e) {  
            System.err.println("Error: " + e.getMessage());  
        }  
        finally {  
            try { socket.close(); } catch (Exception e) {}  
        }  
    }  
}
```


Client-Server-Programmierung

Client

Da der **Client** den Kontakt zum **Server** herstellt, wird die **Adresse** des Services (Rechner + Port) benötigt.

Anschließend wird ein Socket erzeugt und mittels connect die **Verbindung** hergestellt.

Voraussetzung: An der angesprochenen Adresse wartet ein ServerSocket mit accept auf Kontaktaufnahmen.

Danach kann über das Socket **bidirektional** kommuniziert werden.

```
import java.net.InetSocketAddress;
import java.net.Socket;

...

public static void main(String[] args) {

    InetSocketAddress address =
        new InetSocketAddress("localhost", 12345);

    try (Socket socket = new Socket()) {
        socket.connect(address);
        // mit socket: Senden und Empfangen
    }
    catch (Exception e) {
        System.err.println("Error: " + e.getMessage());
    }
}

...
```

Streams

Streams

Was sind Streams?

Unter einem Stream versteht man eine Sequenz von Daten meist unbekannter Länge, die eine unidirektionale "Fließ"-Richtung besitzt.

Die von einem Strom gelieferten Daten können durch ein Programm gelesen und gesammelt werden, um sie anschließend zu verarbeiten ("konsumieren").

Daten Daten Daten
→
(Datenstrom)

Konsumieren

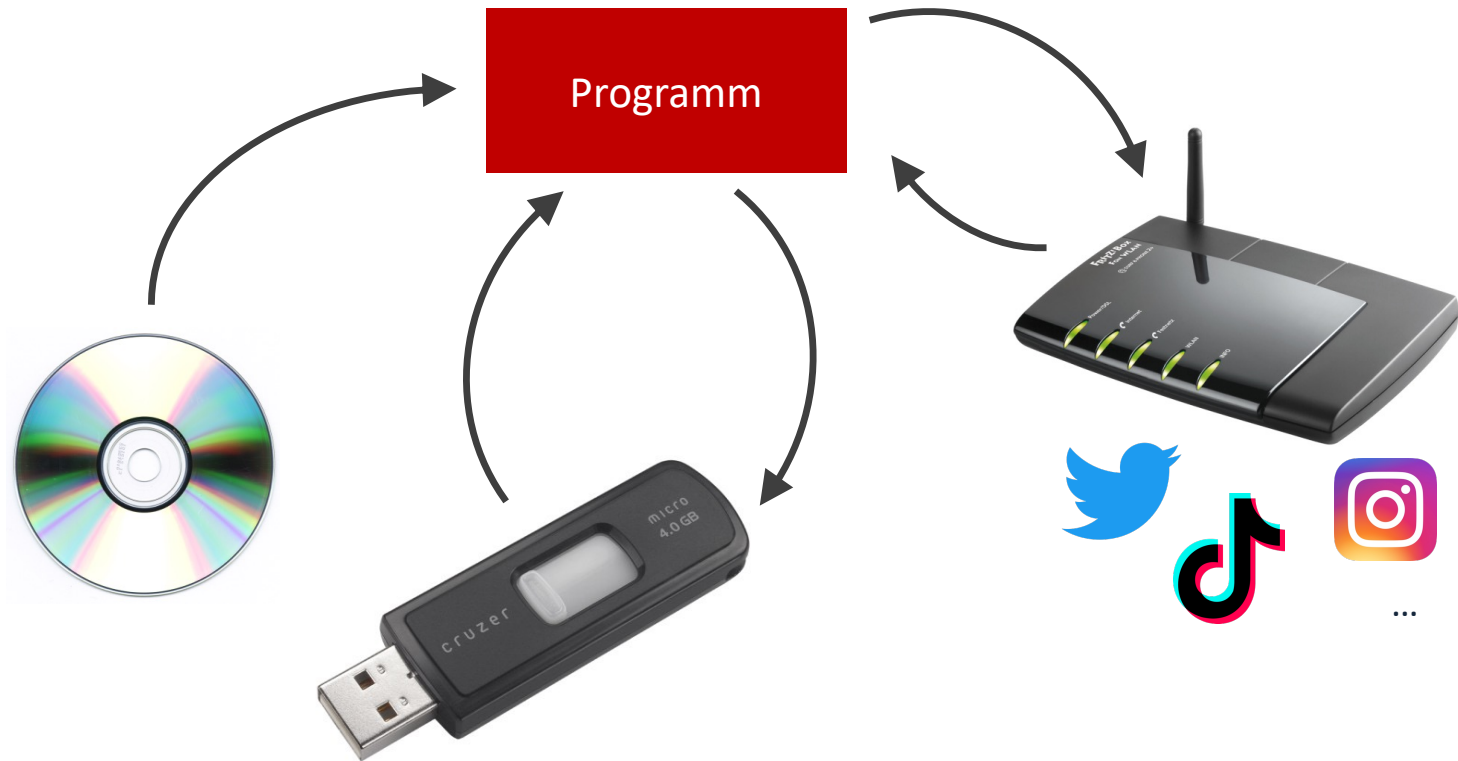
Produzieren

Daten Daten Daten
→
(Datenstrom)

Andererseits kann ein Programm auch ("produzierte") Daten in einen Strom einspeisen bzw. schreiben.

Streams

Was sind Streams?



Streams

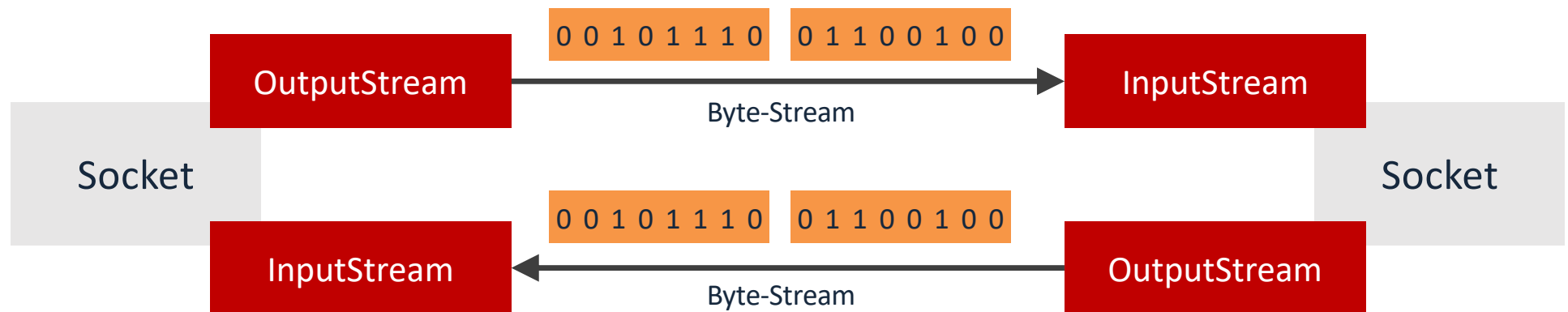
Byte-Streams



- Grundsätzlich werden in einem Stream Bits mit den Werten 0 und 1 übertragen.
- Jeweils acht Bits werden zu einem Byte zusammengefasst.
- **Quellen** sind Unterklassen von `OutputStream`, **Senken** sind Unterklassen von `InputStream`.

Streams

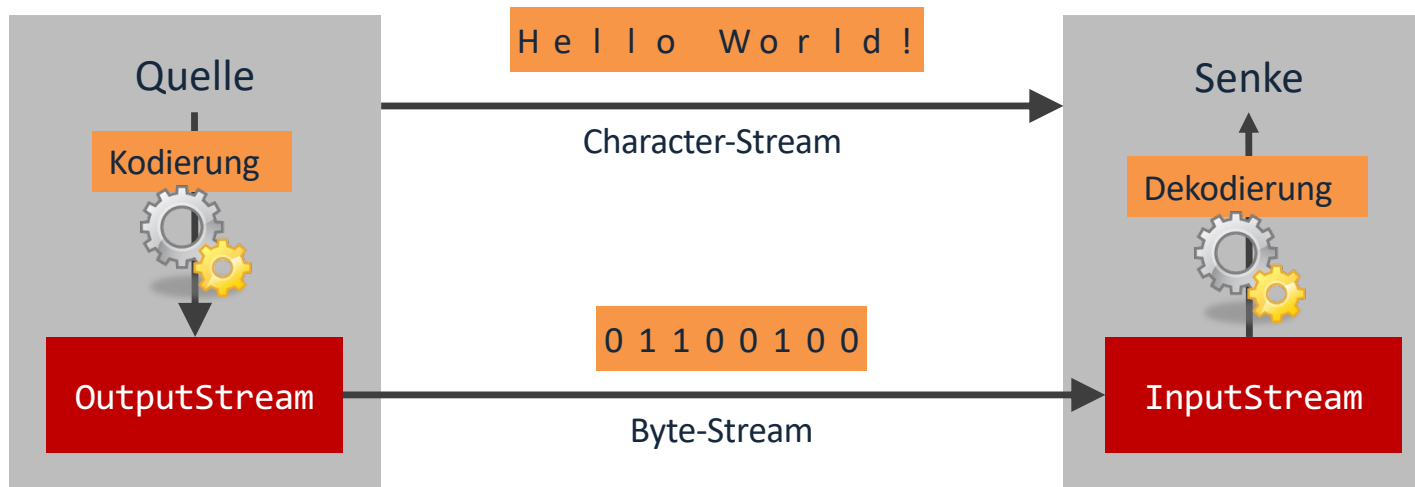
Socket und Byte-Streams



- Jedes Socket besitzt einen OutputStream, über den es Daten versenden kann.
- Jedes Socket besitzt einen InputStream, über den es Daten empfangen kann.
- Bei zwei verbundenen Sockets sind diese Streams wechselseitig verknüpft.

Streams

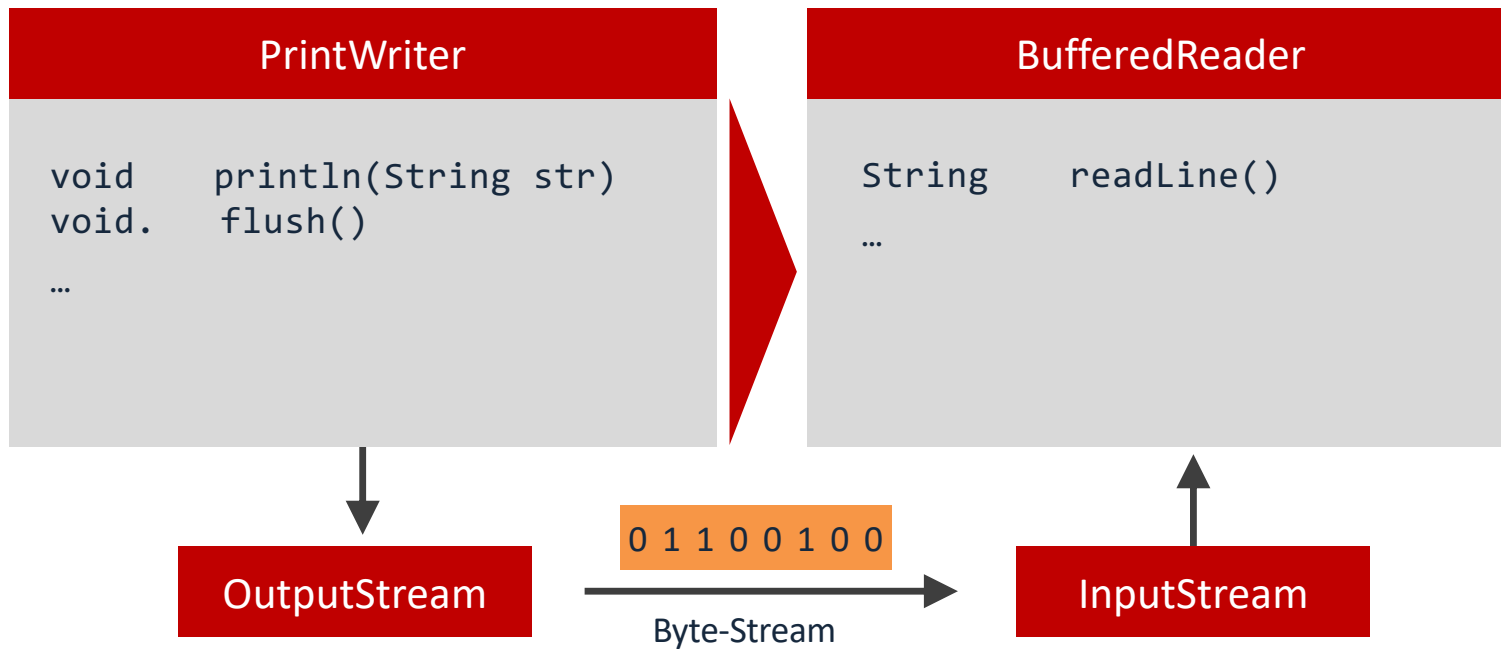
Character-Streams



Um das Senden und Empfangen von Zeichen und Zeichenketten zu vereinfachen, unterstützt Java Character-Streams.

Streams

PrintWriter und BufferedReader



Streams

Senden einer Textzeile über ein Socket

- Für das Versenden von Nachrichten besitzt jedes Socket einen `OutputStream`, über den einzelne Bytes verschickt werden können.
- Möchte man zeilenweise Nachrichten senden, bietet es sich an, einen `PrintWriter` mit dem `OutputStream` zu verknüpfen, mit dem Zeilen mittels `println` ausgegeben werden.
- Die Übertragung findet meist nicht sofort statt, sondern es werden Nachrichten gesammelt. Mit `flush` kann man aber die Ausgabe erzwingen.

```
public static void sendLine(Socket socket, String line) throws IOException {  
    OutputStream out = socket.getOutputStream();  
    PrintWriter writer = new PrintWriter(out);  
    writer.println(line);  
    writer.flush();  
}
```

Streams

Empfangen einer Textzeile über ein Socket

- Für das Empfangen von Nachrichten besitzt jedes Socket einen `InputStream`, über den einzelne Bytes entgegengenommen werden können.
- Möchte man zeilenweise Nachrichten empfangen, bietet es sich an, einen `BufferedReader` mit dem `InputStream` zu verknüpfen, mit dem Zeilen mittels `readLine` ausgelesen werden.

```
public static String receiveLine(Socket socket) throws IOException {  
    InputStream in = socket.getInputStream();  
    BufferedReader reader = new BufferedReader(new InputStreamReader(in));  
    return reader.readLine();  
}
```

Vielen Dank für die Aufmerksamkeit!

Fragen?